

Le basi

ALTER TABLE

Come cambiare una tabella esistente senza ricrearla da zero.

I progetti cambiano, e le tabelle con loro

È raro che una struttura resti perfetta per sempre. All'inizio di un progetto fai una prima scelta. Poi arrivano nuove esigenze, nuovi campi, nomi migliori o regole più precise.

`ALTER TABLE` serve proprio a modificare una tabella che esiste già.

Aggiungere una colonna

Uno degli usi più comuni è questo:

```
ALTER TABLE clienti
ADD telefono VARCHAR(20);
```

Qui stai dicendo al database di aggiungere una nuova colonna chiamata `telefono`.

Rinominare una colonna

A volte il problema non è il contenuto, ma il nome.

```
ALTER TABLE clienti
RENAME COLUMN telefono TO numero_telefono;
```

Con questa modifica rendi la colonna più chiara senza dover ricreare la tabella intera.

Altri cambiamenti possibili

A seconda del database, `ALTER TABLE` può servire anche per:

- eliminare una colonna
- cambiare un tipo di dato
- aggiungere un vincolo
- togliere una regola esistente
- rinominare la tabella

L'idea centrale resta sempre la stessa: stai facendo manutenzione sulla struttura.

Perché conviene essere prudenti

Quando modifichi una tabella vuota, il rischio è basso. Quando modifichi una tabella piena di dati, la faccenda si fa più delicata.

Per esempio, cambiare tipo a una colonna può creare problemi se i vecchi valori non si adattano bene. Eliminare una colonna significa perdere tutti i dati che conteneva.

Una buona abitudine operativa

Prima di fare un `ALTER TABLE`, è utile:

1. capire bene l'effetto della modifica
2. verificare se altre query o applicazioni dipendono da quella tabella
3. provare la modifica in un ambiente di test
4. fare un backup quando il contesto lo richiede

:::caution `ALTER TABLE` è indispensabile nella vita reale, ma proprio perché tocca la struttura conviene usarlo con metodo. :::

CREATE DATABASE

Come creare il contenitore principale che ospiterà tabelle e dati.

Prima esiste il contenitore, poi il contenuto

Un database è il contenitore generale dei tuoi dati. Prima di creare tabelle, colonne e righe, devi avere un posto in cui tutto questo possa vivere.

È simile a quando prepari una cartella principale prima di riempirla di documenti e sottocartelle.

La sintassi più semplice

```
CREATE DATABASE negozio;
```

Con questa istruzione chiedi al server di creare un database chiamato `negozio`.

Cosa NON fa questo comando

CREATE DATABASE non crea ancora tabelle e non salva ancora clienti, prodotti o ordini. Prepara solo il contenitore vuoto.

Dopo questo comando, di solito il passo successivo è collegarti a quel database e iniziare a definirne la struttura.

Quando si usa davvero

Questo comando compare spesso quando:

- inizi un progetto nuovo
- prepari un ambiente di sviluppo o test
- separi i dati di più applicazioni

In molte situazioni quotidiane viene eseguito poche volte. Dopo la fase iniziale, il lavoro si sposta soprattutto su tabelle e query.

Una differenza pratica tra sistemi

Non tutti i database trattano questo comando nello stesso modo. In PostgreSQL e MySQL è del tutto normale creare database con un comando esplicito.

In SQLite, invece, il database spesso coincide con un file. Il concetto è simile, ma il modo pratico di crearlo cambia.

Da portare a casa

Ricorda questa idea: **il database è la casa, le tabelle sono le stanze**. Con `CREATE DATABASE` stai costruendo la casa, non stai ancora arredando le stanze.

CREATE TABLE

Come definire una tabella, le sue colonne e le prime regole di base.

Creare la struttura di una tabella

`CREATE TABLE` serve a creare una nuova tabella.

Non stai ancora inserendo dati. Stai preparando lo spazio che li conterrà: scegli il nome della tabella, le colonne e alcune regole di base.

È come preparare un modulo prima di farlo compilare. Decidi quali campi servono: nome, email, data di iscrizione.

Un primo esempio

Questa query crea una tabella per salvare clienti:

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(150),  
  data_iscrizione DATE  
);
```

Dopo questa query, la tabella `clienti` esiste, ma è vuota. Per aggiungere righe userai `INSERT`.

Leggiamola con calma

Dentro le parentesi trovi le colonne:

- `id` è un numero intero e identifica ogni cliente.

- `nome` è testo fino a 100 caratteri e non può mancare.
- `email` è testo fino a 150 caratteri.
- `data_iscrizione` contiene una data.

La tabella, tradotta in italiano, dice:

"Ogni cliente ha un id, un nome, forse un'email e forse una data di iscrizione."

Quando una tabella si riesce a leggere così, sei sulla strada giusta.

La chiave primaria

PRIMARY KEY indica la **chiave primaria**, cioè il valore che identifica una riga senza confonderla con altre.

È come il numero di tessera di una palestra. Due persone possono chiamarsi entrambe Luca, ma non dovrebbero avere lo stesso numero di tessera.

Per questo spesso si usa una colonna `id` : non descrive la persona, la identifica.

Colonne obbligatorie con **NOT NULL**

NOT NULL significa che una colonna non può restare vuota.

Nel nostro esempio:

```
nome VARCHAR(100) NOT NULL
```

vuol dire che ogni cliente deve avere un nome.

Suggerimento: Usa **NOT NULL** per i dati davvero necessari. Se un'informazione può mancare nella vita reale, non renderla obbligatoria solo per abitudine.

Tipi e regole

Quando crei una colonna, scegli almeno due cose:

- il **tipo**, cioè che forma avrà il dato
- le **regole**, cioè cosa il database deve controllare

Per esempio:

```
prezzo DECIMAL(10, 2) NOT NULL
```

Qui `DECIMAL(10, 2)` dice che `prezzo` è un numero decimale. `NOT NULL` dice che il prezzo deve esserci.

Il database non conserva soltanto i dati. Ti aiuta anche a tenerli ordinati e coerenti.

Scegliere colonne chiare

All'inizio può venire voglia di creare colonne generiche come `info` o `dati`.

Meglio evitarlo. Una tabella diventa più utile quando ogni colonna ha un compito chiaro.

```
CREATE TABLE prodotti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  prezzo DECIMAL(10, 2) NOT NULL  
);
```

Qui capisci subito cosa contiene ogni colonna. Non devi indovinare.

Prima di creare una tabella

Fai queste domande:

- che cosa rappresenta ogni riga?
- quale colonna identifica una riga?
- quali dati sono obbligatori?
- quali dati possono mancare?
- i tipi scelti sono adatti ai valori reali?

Non serve progettare tutto perfettamente al primo colpo. Però queste domande ti evitano molte tabelle confuse.

Riepilogo rapido

- `CREATE TABLE` crea la struttura di una tabella.
 - Ogni colonna ha un nome e un tipo.
 - `PRIMARY KEY` identifica ogni riga.
 - `NOT NULL` rende una colonna obbligatoria.
 - Dopo aver creato la tabella, puoi aggiungere dati con `INSERT`.
-

Tipi di dati in SQL

Come scegliere il tipo giusto per numeri, testi, date e valori logici.

Ogni colonna ha bisogno della forma giusta

Quando crei una colonna, devi decidere che cosa potrà contenere. È come scegliere il contenitore corretto prima di sistemare gli oggetti in cucina.

Un barattolo per il sale non è la scelta migliore per conservare la pasta. Allo stesso modo, una colonna pensata per una data non dovrebbe contenere una frase lunga.

Alcuni tipi che userai spesso

Tipo	Significato semplice	Esempio
<code>INT</code>	numero intero	<code>42</code>
<code>VARCHAR(100)</code>	testo corto	<code>'Luca'</code>
<code>TEXT</code>	testo lungo	una descrizione
<code>DATE</code>	data	<code>2026-04-24</code>
<code>BOOLEAN</code>	vero o falso	<code>TRUE</code>
<code>DECIMAL(10,2)</code>	numero decimale preciso	<code>19.99</code>

Perché la scelta conta davvero

Il tipo di dato non è un dettaglio estetico. Aiuta il database a capire come salvare l'informazione, come confrontarla e come difenderla da errori banali.

Per esempio, una colonna prezzo va trattata in modo diverso da una colonna nome. Una colonna data si comporta in modo diverso da un testo normale.

VARCHAR e TEXT : sembrano simili, ma non sono uguali

Quando inizi è normale pensare: "se entrambi tengono testo, perché esistono due tipi?"

- `VARCHAR(100)` dice: **qui entra testo, ma con un limite di lunghezza**
- `TEXT` dice: **qui può entrare anche testo molto lungo**

Per un nome o un'email, spesso `VARCHAR` è una scelta naturale. Per una recensione o una descrizione dettagliata, `TEXT` può avere più senso.

Numeri interi o numeri con virgola

INT è comodo per quantità, contatori e identificatori. Se devi salvare quante unità hai in magazzino, è un buon candidato.

DECIMAL è più adatto per prezzi, importi e valori dove la precisione conta. Quando ci sono di mezzo soldi, conviene essere precisi.

Un esempio dentro una tabella

Qui sotto vedi una tabella dove i tipi di dato raccontano già molto delle colonne:

```
CREATE TABLE prodotti (  
  id INT,  
  nome VARCHAR(100),  
  prezzo DECIMAL(10,2),  
  disponibile BOOLEAN,  
  data_creazione DATE  
);
```


Leggendo questa struttura capisci subito che `prezzo` è un valore numerico preciso, `disponibile` è un sì/no e `data_creazione` è una data.

...tip Se sei indeciso sul tipo, chiediti: "che aspetto ha questo dato nel mondo reale?" La risposta spesso ti guida bene.
...

DROP TABLE

Come eliminare completamente una tabella e quando farlo con cautela.

Qui non stai riordinando: stai togliendo tutto

DROP TABLE elimina una tabella intera. Non cancella solo le righe. Fa sparire anche la struttura della tabella.

È come gettare via un intero raccoglitore, non solo i fogli che contiene.

Il comando base

```
DROP TABLE clienti;
```

Dopo questo comando la tabella `clienti` non esiste più.

Cosa viene eliminato davvero

Con **DROP TABLE** perdi:

- il nome della tabella
- le sue colonne
- i dati salvati dentro
- eventuali collegamenti o dipendenze da gestire

Per questo è molto diverso da **DELETE**, che toglie righe ma lascia viva la tabella.

Quando può avere senso

Ci sono casi in cui è una scelta legittima:

- hai creato una tabella di prova
- una tabella temporanea non serve più
- stai rifacendo lo schema in ambiente di sviluppo

In un database importante o condiviso, però, questo comando richiede molta attenzione.

Le domande da farti prima

Prima di lanciare il comando, fermati e controlla:

1. voglio davvero eliminare anche la struttura?
2. ci sono altre tabelle che dipendono da questa?
3. esiste un backup o un piano di recupero?

Queste verifiche ti proteggono da errori costosi.

:::caution Se il tuo obiettivo è solo svuotare la tabella ma lasciarla esistente, `DROP TABLE` non è lo strumento giusto. :::

Database relazionali

Cosa sono tabelle, righe, colonne e relazioni tra i dati.

Immaginalo come un insieme di fogli collegati

Un database relazionale assomiglia a una raccolta di tabelle. Se hai mai visto un foglio Excel, hai già un'immagine utile in testa.

La differenza importante è questa: qui le tabelle possono collegarsi tra loro in modo ordinato.

I tre pezzi da riconoscere subito

Dentro una tabella trovi sempre:

- una **tabella**: l'intero elenco, per esempio `clienti`
- una **riga**: un elemento preciso, per esempio un cliente

- una **colonna**: un tipo di informazione, per esempio `nome` o `email`

Guarda questa mini tabella:

id	nome	citta
1	Luca	Roma
2	Marta	Milano

Qui ogni riga è una persona. Ogni colonna descrive un aspetto di quella persona.

Perché si usa la parola relazionale

Perché le tabelle possono fare riferimento una all'altra. Per esempio puoi avere:

- una tabella `clienti`
- una tabella `ordini`

Ogni ordine può contenere un collegamento al cliente che lo ha effettuato. In questo modo i dati restano più ordinati e non devi riscrivere tutto ogni volta.

Un'analogia che aiuta davvero

Pensa a una biblioteca.

- un foglio tiene i **lettori**
- un altro foglio tiene i **prestiti**

Nei prestiti non hai bisogno di riscrivere sempre l'indirizzo completo del lettore. Ti basta un riferimento al lettore giusto. **Questo è il cuore del modello relazionale: collegare bene le informazioni senza duplicarle troppo.**

Cosa rende utile questo modello

Un database relazionale aiuta a:

- mantenere i dati più puliti
- evitare ripetizioni inutili
- cercare informazioni con precisione
- combinare dati di tabelle diverse
- applicare regole per ridurre gli errori

Un'anticipazione dei prossimi passi

Più avanti incontrerai termini come `PRIMARY KEY`, `FOREIGN KEY` e `JOIN`. Non serve padroneggiarli subito.

Per ora ti basta questa immagine: **un database relazionale è un insieme di tabelle che collaborano tra loro in modo controllato.**

Cos'è SQL

Una guida semplice per capire cosa sia SQL, perché esiste e in quali situazioni lo usi davvero.

Partiamo da una scena concreta

Immagina di avere un negozio online. Ci sono clienti che comprano, prodotti in vendita, ordini fatti, pagamenti ricevuti, indirizzi per spedire. Tutte queste informazioni devono essere conservate da qualche parte, in modo ordinato e facile da cercare.

Quel "posto" si chiama database. E **SQL è il linguaggio per parlare con quel posto**, per chiedergli le informazioni che ti servono e per aggiungerne di nuove.

:::note Se non hai mai sentito la parola "database", niente paura. Pensa a un armadio con tanti cassetti numerati. Ogni cassetto contiene una cosa diversa: uno ha i nomi dei clienti, un altro ha i prodotti, un altro ancora gli ordini. Il database è proprio così, ma al posto dei fogli di carta usa i dati nel computer. :::

Cosa significa il nome

SQL sta per **Structured Query Language**. Il nome può sembrare impegnativo, ma l'idea è semplice: è un linguaggio fatto per fare richieste strutturate a un insieme di dati.

Una *query* è una richiesta. **Pensa a una query come a una domanda specifica** che fai a un amico: "Quali sono i clienti di Roma?" oppure "Dov'è quel prodotto?". La differenza è che invece di chiedere a un amico, la fai al database.

:::note "Strutturato" significa che i dati sono organizzati in modo ordinato, come una tabella con righe e colonne, non sparsi ovunque. :::

Cosa puoi fare con SQL

Con SQL puoi fare quattro cose principali, come se stessi lavorando su un archivio:

- **leggere** dati già salvati (come cercare un libro in una biblioteca)
- **inserire** nuovi dati (come aggiungere un nuovo libro allo scaffale)
- **aggiornare** dati esistenti (come correggere un errore di ortografia in una scheda)
- **eliminare** dati che non servono più (come buttare un vecchio catalogo)

E poi c'è anche il **collegare** dati che stanno in tabelle diverse — come unire la lista dei clienti con la lista degli ordini per vedere cosa hanno comprato.

:::note In gergo tecnico, le "tabelle" sono come fogli Excel: hanno righe e colonne. Ogni riga è un elemento (un cliente, un prodotto), ogni colonna è un tipo di informazione (nome, prezzo, data). :::

Dove lo incontri nella pratica

SQL è molto più comune di quanto sembri. Lo trovi dietro a tanti strumenti che usi ogni giorno, anche se non te ne accorgi:

- **siti di e-commerce** quando cerchi un prodotto
- **app di prenotazione** quando controlli la disponibilità di un hotel
- **gestionali aziendali** quando cerchi un cliente
- **software bancari** quando controlli il saldo
- **pannelli di amministrazione** quando gestisci un sito web
- **strumenti di analisi** quando generi un report

:::tip La prossima volta che fai una ricerca su un sito e appare un risultato, pensa: "Qualcuno ha fatto una query SQL per trovare questo!" :::

Quando un'app salva o recupera dati strutturati, c'è una buona possibilità che in qualche punto stia usando SQL.

Un esempio piccolo da leggere senza paura

Ora vediamo un esempio pratico. Questa query legge una colonna precisa di una tabella:

```
SELECT nome
FROM clienti;
```

Cosa significa in parole semplici: "Prendi la colonna `nome` dalla tabella `clienti`".

- `SELECT` = "prendi" o "mostra"
- `nome` = la colonna che vuoi vedere
- `FROM` = "da" (indica da dove prendere i dati)
- `clienti` = la tabella che contiene i dati

:::note Le parole chiave come SELECT e FROM sono sempre in maiuscolo per convenzione, ma SQL non è sensibile alle maiuscole. Puoi scriverle anche in minuscolo. :::

Hai appena letto il tuo primo comando SQL. È un piccolo passo, ma è già il cuore di tutto: fare una domanda chiara ai dati.

Perché vale la pena impararlo

Capire SQL ti rende più forte anche quando usi strumenti più comodi. Se un giorno lavorerai con ORM, dashboard o framework, sapere SQL ti aiuterà a capire cosa succede davvero sotto la superficie.

:::tip Quando una query ti sembra oscura, prova a trasformarla in una frase italiana. Spesso la nebbia si dirada subito. :::

Riassunto veloce

SQL è come un linguaggio per parlare con un database. Lo usi per chiedere informazioni, aggiungerne di nuove, modificarle o cancellarle. E non devi essere un esperto per iniziare: basta conoscere le basi per fare le tue prime richieste.